

專題計畫書

成員：郭湘穎

一、 研究動機與背景

隨著物聯網（IoT）與人工智慧技術的普及，智慧零售與智慧餐飲（Smart Retail & Catering）已成為現代商業轉型的重要趨勢。在眾多餐飲業態中，咖啡廳具有高度的「空間依賴性」與「客群多樣性」。然而，現今多數連鎖咖啡廳或獨立店家的營運決策與商品推薦，仍停留在傳統的靜態菜單或固定時段促銷，無法根據店內的即時環境動態做出靈活調整。傳統咖啡廳營運主要面臨兩大痛點：

1. 空間利用與人流波動大：咖啡廳在不同時段的人流差異極大。尖峰時段湧入大量外帶或急需提神的上班族；而離峰時段則多為尋找安靜空間的文青、商務客或偏好特定時段活動的山型人。

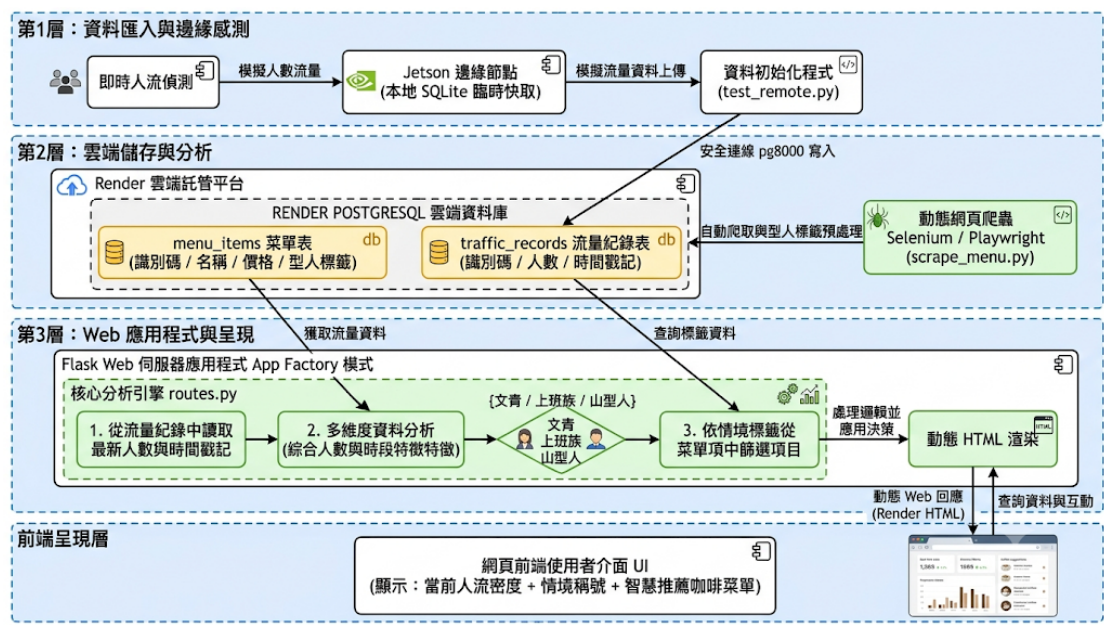
2. 缺乏即時情境感知推薦：當店內擁擠時，若系統仍推薦製作繁複、適合久坐的單品，將大幅增加製作負擔；反之，若在空曠安靜時，未能及時推薦高客單價、適合慢飲的享受型飲品，則錯失了優化顧客體驗的機會。

因此，本專案期望提出一個結合邊緣運算、雲端架構與數理分析的智慧解決方案。透過前端設備即時感知店內人流，並由後端系統利用數學變化率模型動態分析「顧客型人情境」，落實數據驅動的「即時情境化餐點推薦」。

二、 研究目的

實現邊緣至雲端（Edge-to-Cloud）的即時數據整合：利用邊緣運算高即時性、低延遲的優勢，即時感測並估算咖啡廳內的當前人數，並將數據即時同步至遠端 Render PostgreSQL 雲端資料庫，建立高可靠性的環境數據流。建構自動化動態網路數據爬取機制：開發基於 Selenium 的自動化動態網路爬蟲腳本（scrape_menu.py），解決傳統爬蟲無法獲取 JavaScript 渲染內容的問題，定期收集連鎖咖啡廳的最新餐點資訊與定價，並寫入資料庫建立結構化標籤。結合數學專業實作「動態型人決策模型」：捨棄傳統靜態的「固定人數門檻」，導入微積分中導數與變化率的概念分析即時人流。當 Flask 後端系統讀取數據後，會計算人流密度與隨時間的變化率，精準判別「文青」、「上班族」與「山型人」情境，動態推薦最適合的商品。

三、系統架構



Architecture Explanation (系統架構對照表)

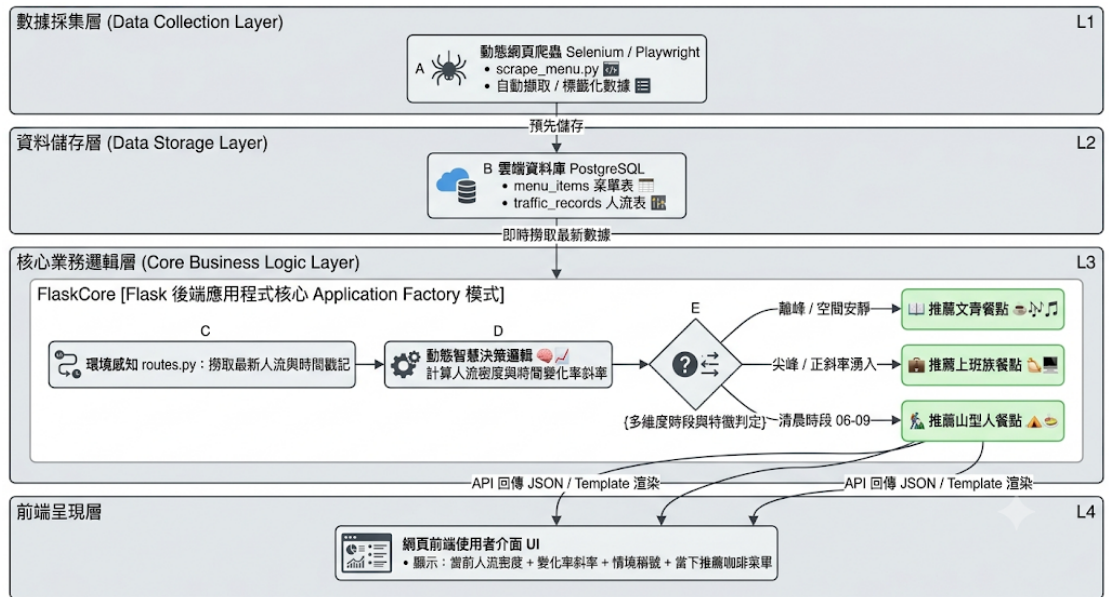
Component (元件)	Description (功能說明)
Flask Frontend & Backend	採用 Python Flask 輕量化網頁框架。前端負責即時渲染當下情境與推薦餐點；後端 (Application Factory 模式) 作為核心業務邏輯中心，負責串接資料庫與數學決策模型。
Edge-to-Cloud Integration	邊緣端定時將實際人數數據，透過輕量化連線 (pg8000) 寫入雲端。專案全面使用 .env 進行環境變數隔離，確保資料庫連線字串等機密資訊不外流。
Automated Menu Scraper	內建 Selenium 動態網頁爬取腳本，負責模擬真實瀏覽器行為收集咖啡廳商品資料，並依據商品特性自動關聯「型人標籤」。
PostgreSQL Database	採用 Render PostgreSQL 雲端資料庫。包含 menu_items (儲存爬蟲分類好的菜單與標籤) 與 traffic_records (儲存即時回傳的人流軌跡與時間戳記)。

Deployment Infrastructure	程式碼由 GitHub 版本控制管理。透過 requirements.txt 嚴格定義套件版本，結合 Gunicorn WSGI 部署於 Render 雲端生產環境。
---------------------------	---

四、開發環境與技術

項目 (Item)	實際開發環境與規格 (Specifications)
作業系統 (OS)	macOS / Windows 11 (開發端) ； Linux (Render 雲端部署端)
程式語言 (Language)	Python 3.13+
網頁後端框架 (Framework)	Flask 3.1.3
資料庫系統 (Database)	PostgreSQL (託管於 Render 雲端平台)
資料庫驅動 (DB Driver)	pg8000 1.31.5 (純 Python 實作之輕量化 PostgreSQL 驅動)
網路數據爬取 (Scraper)	Selenium (用於解析並擷取需 JavaScript 動態渲染的餐點資料)
版本控制與協作	GitHub

五、系統功能設計圖和解說



1. GitHub 專案與協同合作

- 實作內容：專案完整導入 GitHub 進行原始碼版本控制與程式碼管理（Repository 網址：<https://github.com/doris40421-cloud/smart-cafe-analyzer.git>）。
- 專案架構：採用 Python Flask 的應用程式工廠模式（Application Factory Pattern），將路由（routes.py）、資料庫模型與爬蟲腳本進行模組化解耦，建立清晰的專案目錄結構。
- 規範管理：於根目錄透過 requirements.txt 嚴格管理第三方套件版本，並設定 .gitignore 排除環境變數檔案（.env），確保專案符合雲端部署規範。

2. PostgreSQL 雲端資料庫

- 實作內容：後端資料庫託管於 Render 雲端空間的 PostgreSQL 關聯式資料庫。
- 資料表設計：
 - * menu_items：儲存由爬蟲定期抓取、清洗並分類好的咖啡廳菜單、價格與型人情境標籤。
 - traffic_records：儲存由邊緣端（或模擬腳本）定時上傳的店內即時人流數量與精確時間戳記（Timestamp）。
- 安全連線：採用純 Python 實作的輕量化 pg8000 驅動進行後端與雲端資料庫的連線，並利用環境變數檔案（.env）隔離保護資料庫連線字串，避免機密資訊外流。

3. 動態網頁爬蟲（Selenium / Playwright）

- 實作內容：針對現代咖啡廳官網多採用 JavaScript 動態渲染、導致傳統靜態請求（Requests）無法抓取資料的問題，本專案開發獨立的動態爬蟲腳本 scrape_menu.py。
- 技術應用：整合 Selenium / Playwright 自動化測試框架，模擬真實瀏覽器載入行為並解析動態 DOM 結構，自動化擷取最新餐點名稱與定價。
- 資料預處理：抓取資料後進行清洗，依據商品特性與出餐速度（例如：手沖單品 vs 快速拿鐵），自動關聯並分類貼上「文青」、「上班族」與「山型人」的初始型人標籤，隨後寫入雲端 PostgreSQL。

4. 資料分析

資料分析與智慧決策模組：後端（routes.py）即時撈取最新人流數據與時間戳記，利用數學變化率模型（導數概念）進行多維度環境感知情境判定，精準識別『文青』、『上班族』與『山型人』三大客群，動態過濾出最符合當下狀態的咖啡品項。

六、未來期望

本專案目前已成功串接邊緣端數據輸入、Render 雲端 PostgreSQL 儲存與 Flask 動態情境推薦，初步驗證了結合數學模型的智慧餐飲推薦可行性。針對未來演進，規劃以下方向：

1. 實體邊緣端電腦視覺（CV）整合：未來期望將專案部署至實體 NVIDIA Jetson 裝置，結合 YOLO 輕量化物件偵測或人體骨架追蹤技術，實時計算店內人數並回傳數據，取代目前的模擬腳本。
2. 導入黎曼積分與進階時序預測：在累積足夠的 traffic_records 後，將引入更嚴謹的數值分析方法。透過計算特定時間區間的下積分（Lower Sums）來精確評估離峰時段的最低容客量，並導入 ARIMA 或 LSTM 機器學習模型，不僅看「現在」，更能「預測未來」的客流量，輔助店家進行精準庫存管理。
3. 擴展多維度環境變數：進一步串接外部環境 Web API（如天氣、氣溫）。當外在天氣與店內擁擠度之變化率產生共振時，提供更具深度的動態定價策略，打造全方位的智慧零售生態。